

Understanding the Full GraphRAG Pipeline

The big picture — what are we building, and why?

Normal RAG (Retrieval-Augmented Generation) works like this: chop documents into chunks, turn each chunk into a vector embedding, and when a question comes in, find the chunks whose embeddings are most similar (cosine similarity) to the question's embedding.

GraphRAG is a different approach. Instead of just retrieving similar-sounding chunks, it builds a **knowledge graph** — entities (people, companies, legal cases) connected by labeled relationships (SUED, FILED_AGAINST, WORKS_FOR) — then groups that graph into topic clusters, summarizes each cluster, and answers questions using those summaries. This is better suited to broad, connect-the-dots questions like "what's going on with AI copyright disputes overall?" — something a single similar-sounding chunk wouldn't answer well.

There are **five stages** to this pipeline, each one covered below in order:

1. **Extraction** — `GraphRAGExtractor` : read each chunk of text, ask an AI to find entities/relationships.
2. **Wiring it together** — `PropertyGraphIndex` : actually run the extraction across every chunk and build the graph.
3. **Clustering + summarizing** — `GraphRAGStore` : group the graph into topic clusters, summarize each one.
4. **Answering questions** — `GraphRAGQueryEngine` : use those summaries to answer a user's question.
5. **Saving and visualizing** — persist the finished graph to disk, and export it as an interactive webpage.

Stage 1: `GraphRAGExtractor` — pulling facts out of each chunk

Goal: for every small chunk of text, ask an AI "who/what is mentioned here, and how are they connected?" and turn the answer into structured graph data.

Before this stage runs, your raw documents have already been split into small chunks of text (this happens elsewhere, earlier in the pipeline — not part of this class).

Step 1 — Process many chunks at once

```
async def acall(self, nodes, show_progress=False, **kwargs):
    jobs = [self._aextract(node) for node in nodes]
    return await run_jobs(jobs, workers=self.num_workers, ...)
```

Instead of handling chunks one at a time, up to **4 chunks are processed simultaneously** (`num_workers=4`). This works because most of the time here is spent *waiting* on the AI's response over the network — waiting on 4 things at once is much faster than waiting on them one by one.

Step 2 — Get the chunk's text and build a prompt

```
text = node.get_content(metadata_mode="llm")

base_prompt = self.extract_prompt.format(
    text=text, max_knowledge_triplets=self.max_paths_per_chunk
)
```

Pulls the plain text out of one chunk, and fills in a template asking the AI to extract entities/relationships (capped at 10 per chunk).

```
json_instructions = """
Respond with ONLY valid JSON in exactly this format...
{
  "entities": [...],
  "relationships": [...]
}"""
full_prompt = base_prompt + json_instructions
```

Strict formatting instructions get appended, **with a concrete example**, because just describing the format in words isn't reliable enough on its own — models follow examples more consistently than descriptions.

Step 3 — Ask the AI, with retries

```
for attempt in range(max_retries):    # tries up to 3 times
    await asyncio.sleep(4)
    current_llm = get_next_llm()
    response = await current_llm.acomplete(full_prompt)
    raw = response.text.strip()
```

- Waits 4 seconds before each try, so 4 concurrent chunks don't all hit the AI's rate limit at the same instant.
- Grabs a different API key each time from a rotating pool, so a retry isn't stuck reusing a key that's already blocked.
- Sends the prompt and gets the raw text answer back.

Step 4 — Clean up messy formatting

```
if raw.startswith("`"):
    raw = raw.strip("`")
    if raw.startswith("json"):
        raw = raw[4:]
    ...
```

Als often wrap their answer in markdown code blocks even when explicitly told not to. This strips all that wrapping off, leaving pure JSON text.

Step 5 — Parse and validate

```
parsed = json.loads(raw)
result = ExtractionResult.model_validate(parsed)
entities      = result.entities
relationships = result.relationships
break
```

Two checks happen: 1. **Is this valid JSON at all?** (`json.loads`) 2. **Does it have the right fields/ shape?** (`model_validate`)

Only if both pass does the loop stop early with `break` . If anything fails, it logs the error and retries — up to 3 total attempts. If all 3 fail, the chunk just ends up with nothing extracted, instead of crashing the whole run.

Step 6 — Turn entities into graph nodes

```
existing_nodes += [
    EntityNode(
        name=entity.name,
        label=entity.type,
        properties={**base_metadata, "entity_description": entity.description},
    )
    for entity in entities
]
```

Every extracted entity (e.g. "Apple Inc.") becomes a proper node object with a name, category, and source metadata attached — so you can always trace it back to where it came from.

Step 7 — Turn relationships into graph edges

```
for rel in relationships:
    source_node = EntityNode(name=rel.source, label=entity_lookup.get(rel.source, "ENTITY"), ...)
    target_node = EntityNode(name=rel.target, label=entity_lookup.get(rel.target, "ENTITY"), ...)
    ...
    existing_relations.append(Relation(
        label=rel.relation,
        source_id=source_node.id,
        target_id=target_node.id,
        properties={**base_metadata, "relationship_description": rel.description},
    ))
```

Builds the connecting "edge" between two entities (e.g. Apple → FILED_AGAINST → Samsung), avoiding duplicate nodes for entities already added in Step 6.

Step 8 — Save it back and return

```
node.metadata[KG_NODES_KEY] = existing_nodes
node.metadata[KG_RELATIONS_KEY] = existing_relations
return node
```

The chunk is returned carrying its own mini graph of facts, hidden in its metadata. This repeats independently for every chunk.

Stage 2: Wiring it together with `PropertyGraphIndex`

Everything defined in Stage 1 is just a *blueprint* until this stage — this is where it actually gets configured and **run**.

```
kg_extractor = GraphRAGExtractor(
    llm=EXTRACTION_LLM,
    extract_prompt=KG_TRIPLET_EXTRACT_TMPL,
    max_paths_per_chunk=MAX_PATHS_PER_CHUNK,
    num_workers=NUM_WORKERS,
)

graph_store = GraphRAGStore()
```

- `kg_extractor` — a real, configured instance of `GraphRAGExtractor`. Note `llm=EXTRACTION_LLM` — a **cheaper/faster** model is deliberately used for extraction, since it runs many times (once per chunk).
- `graph_store` — an empty instance of `GraphRAGStore`, ready to receive nodes and relationships. The comment "GraphRAGStore is both the graph database and the community detection engine" summarizes its dual role well.

```

index = PropertyGraphIndex(
    nodes=nodes,
    kg_extractors=[kg_extractor],
    property_graph_store=graph_store,
    embed_kg_nodes=False,           # we don't need vector similarity search for this GraphRAG pipeline
    show_progress=True,
)

```

`PropertyGraphIndex` is a built-in LlamaIndex class — the orchestrator that actually runs everything:

- `nodes=nodes` — the full list of pre-chunked text nodes.
- `kg_extractors=[kg_extractor]` — a list of extractors to run over every node (just one here, but this design allows chaining more than one).
- `property_graph_store=graph_store` — tells the index to store all resulting graph data into *our* `graph_store`, not some default store.
- `embed_kg_nodes=False` — this is an important one. It tells the index to **skip generating vector embeddings** for the graph nodes entirely. This is exactly why this pipeline doesn't do cosine-similarity search anywhere — relevance later gets decided by directly asking an LLM (see Stage 4), not by comparing vectors. If you wanted to add embedding-based retrieval later, this is the flag you'd flip to `True`.
- `show_progress=True` — shows a progress bar, since this step can take several minutes (it's making many real API calls).

What actually happens when this line runs: every node gets passed through `kg_extractor` (triggering the whole retry/cleanup/validation chain from Stage 1), and the resulting entities/relationships from every chunk get merged into one unified graph, stored inside `graph_store`. By the time this finishes, `graph_store` holds the complete graph built from your entire document collection.

Stage 3: `GraphRAGStore` — clustering and summarizing

Goal: take the full merged graph and group it into topic clusters, then write a short AI summary for each cluster.

Step 1 — Convert to NetworkX format

```

def _to_networkx(self) -> nx.Graph:
    nx_graph = nx.Graph()
    for node in self.graph.nodes.values():
        if isinstance(node, EntityNode):
            nx_graph.add_node(node.id)
    for relation in self.graph.relations.values():
        if relation.source_id in nx_graph and relation.target_id in nx_graph:
            nx_graph.add_edge(relation.source_id, relation.target_id, ...)
    return nx_graph

```

Translates the stored graph into the format the **NetworkX** analysis library understands — just node IDs and connecting lines. NetworkX is a Python library specifically built for analyzing network structures.

Step 2 — Find clusters automatically

```
clusters = hierarchical_leiden(nx_graph, max_cluster_size=MAX_CLUSTER_SIZE)
```

The **Leiden algorithm** looks at the whole web of connections and automatically groups tightly-connected entities into clusters — purely from connection patterns, no manual labeling needed. A size cap stops any one cluster from growing too large and becoming meaningless.

Step 3 — Organize each cluster into readable text

```
def _collect_community_info(self, nx_graph, clusters) -> dict:
    community_mapping = {item.node: item.cluster for item in clusters}
    node_details = { node.id: {"name": ..., "type": ..., "description": ...} for node in ... }
    ...
```

For each cluster, this gathers:

- The readable details of every entity in it.
- Readable sentences describing connections *within* that cluster (e.g. "Apple Inc. -- [FILED_AGAINST]--> Samsung Electronics"), skipping any connection that crosses into a different cluster.

This translation into plain sentences matters because the next step hands everything to an AI, and AI only understands text — not raw graph objects.

Step 4 — Ask the AI to summarize each cluster

```
prompt = f"""You are analysing a cluster of entities from news articles about
AI copyright, governance, and intellectual property.

Entities in this cluster:
{entities_text}

Relationships:
{relationships_text}

Write a concise briefing (3-5 sentences)..."""
```

```

for attempt in range(max_retries):
    time.sleep(4)
    current_llm = get_next_llm()
    response = current_llm.complete(prompt)
    self.community_summaries[community_id] = response.text
    break

```

Same retry pattern as Stage 1 (up to 3 tries, key rotation), but done **one cluster at a time** rather than concurrently — this version uses plain `time.sleep()` instead of `asyncio.sleep()` since there's no concurrency to protect here.

Step 5 — Retrieve the results

```

def get_community_summaries(self) -> dict:
    return self.community_summaries

```

A simple method that hands back the dictionary of all generated summaries, keyed by cluster ID.

Stage 4: `GraphRAGQueryEngine` — answering a user's question

Goal: actually answer a question, using the community summaries from Stage 3. This is the "retrieval + generation" part of Graph RAG.

How relevance is decided — NOT cosine similarity

This is worth calling out explicitly for students coming from normal RAG: **there is no vector search, no embeddings, no cosine similarity anywhere in this pipeline** (confirmed by `embed_kg_nodes=False` back in Stage 2). Instead, relevance is decided by **directly asking an LLM**, in plain English, whether each community summary relates to the question. This only works because clustering already reduced "thousands of chunks" down to "a manageable number of community summaries" — small enough that checking each one with an LLM call is affordable.

The pattern used is **map-reduce**: check every summary individually for relevance ("map"), then combine only the relevant ones into one final answer ("reduce").

Step 1 — Check every community for relevance ("map")

```
def custom_query(self, query_str: str) -> str:
    summaries = self.graph_store.get_community_summaries()

    if not summaries:
        return "No community summaries found. Run graph_store.build_communities() first."

    community_answers = [
        self._answer_from_community(summary, query_str)
        for summary in summaries.values()
    ]
```

For **every single community summary**, ask: "does this cluster relate to the question, and if so, what's the answer based only on this cluster?" Each community is checked completely independently.

```
def _answer_from_community(self, summary: str, query: str) -> str:
    prompt = (
        f"Community summary:\n{summary}\n\n"
        f"Question: {query}\n\n"
        f"If this summary contains information relevant to the question, answer it. "
        f"If not relevant, reply exactly: 'No relevant information.'\n\n"
        f"Answer:"
    )
    response = EXTRACTION_LLM.complete(prompt)
    text = response.text.strip()

    return "" if "no relevant information" in text.lower() else text
```

This uses the **cheaper LLM** (`EXTRACTION_LLM`) — because this call runs once *per community*. With 50 communities, that's 50 API calls for a single question, so an expensive model here would multiply cost fast for a relatively simple task. Asking for one *exact* fallback phrase makes it reliable to detect and filter out afterward.

Step 2 — Filter out anything irrelevant

```
relevant_answers = [a for a in community_answers if a.strip()]

if not relevant_answers:
    return "I don't have enough information in the knowledge graph to answer that question."
```

Any community that came back empty (irrelevant) gets dropped. If nothing in the whole graph was relevant, the user is told honestly instead of getting a made-up answer.

Step 3 — Combine relevant answers into one final answer ("reduce")

```
def _aggregate(self, answers: List[str], query: str) -> str:
    combined = "\n\n---\n\n".join(answers)
    prompt = (
        f"You have received answers from multiple knowledge graph communities about this question:\n\n"
        f"Question: {query}\n\n"
        f"Community answers:\n{combined}\n\n"
        f"Synthesise these into a single, clear, well-structured final answer. "
        f"Remove redundancy, keep all important details, and ensure the answer "
        f"directly addresses the question.\n\n"
        f"Final Answer:"
    )
    return self.llm.complete(prompt).text
```

This runs only **once per question**, so it's worth using the stronger model (`self.llm` , e.g. gpt-4o) for better reasoning quality here.

Why cheap-model-many-times + expensive-model-once?

Step	Runs how often	Model used	Why
Relevance check per community	Once per community (could be dozens)	Cheap/fast (<code>EXTRACTION_LLM</code>)	Simple task, repeated often — cost adds up fast
Final synthesis	Once per question	Strong/expensive (<code>self.llm</code> , e.g. gpt-4o)	Harder reasoning task, but only happens once

Stage 5: Saving and visualizing the graph

Building all of the above is expensive — many LLM calls, several minutes of runtime. This stage is about **not having to redo that work** every time.

Saving the graph with `pickle`

```
import pickle

with open("graph_store.pkl", "wb") as f:
    pickle.dump(graph_store, f)
```

`pickle` converts an entire Python object — here, the whole `graph_store` , including all entities, relationships, and community summaries — into bytes that get written to a file. It can be loaded back later with `pickle.load()` , instantly restoring everything without re-running extraction or clustering.

```
entity_count = sum(1 for n in graph_store.graph.nodes.values() if isinstance(n, EntityNode))
print(f"    Saved graph with {entity_count} entities and 69 communities to disk")
```

Counts how many entities made it into the final graph. **Worth flagging as a teaching point:** the `69` here is hardcoded, not calculated — a more correct version would compute it live with `len(graph_store.community_summaries)`, the same way `entity_count` was computed. Hardcoded "observed" values like this can silently go stale if the pipeline is re-run and the numbers change.

Exporting to JSON for visualization

```
def export_graph_data(graph_store: GraphRAGStore, output_file: str = "graph_data.json"):
    nx_graph = graph_store._to_networkx()
    ...
    graph_data = {
        "nodes": nodes_data,
        "links": links_data,
        "communities": len(graph_store.get_community_summaries()),
    }
    pathlib.Path(output_file).write_text(json.dumps(graph_data, indent=2))
```

This reuses `_to_networkx()` from Stage 3, then reshapes the graph into a simple JSON structure — `nodes`, `links`, and a `communities` count (this time calculated correctly). Each node becomes `{id, label, type, description}` and each edge becomes `{source, target, label, description}` — a format a JavaScript visualization library (D3.js) can read directly.

Building the interactive HTML page

```
def visualize_graph(graph_store=None, graph_data_file="graph_data.json",
                   template_file="graph_template.html", output_file=GRAPH_OUTPUT_FILE):
    if graph_store is not None:
        graph_data = export_graph_data(graph_store, graph_data_file)
    else:
        graph_data = json.loads(pathlib.Path(graph_data_file).read_text())

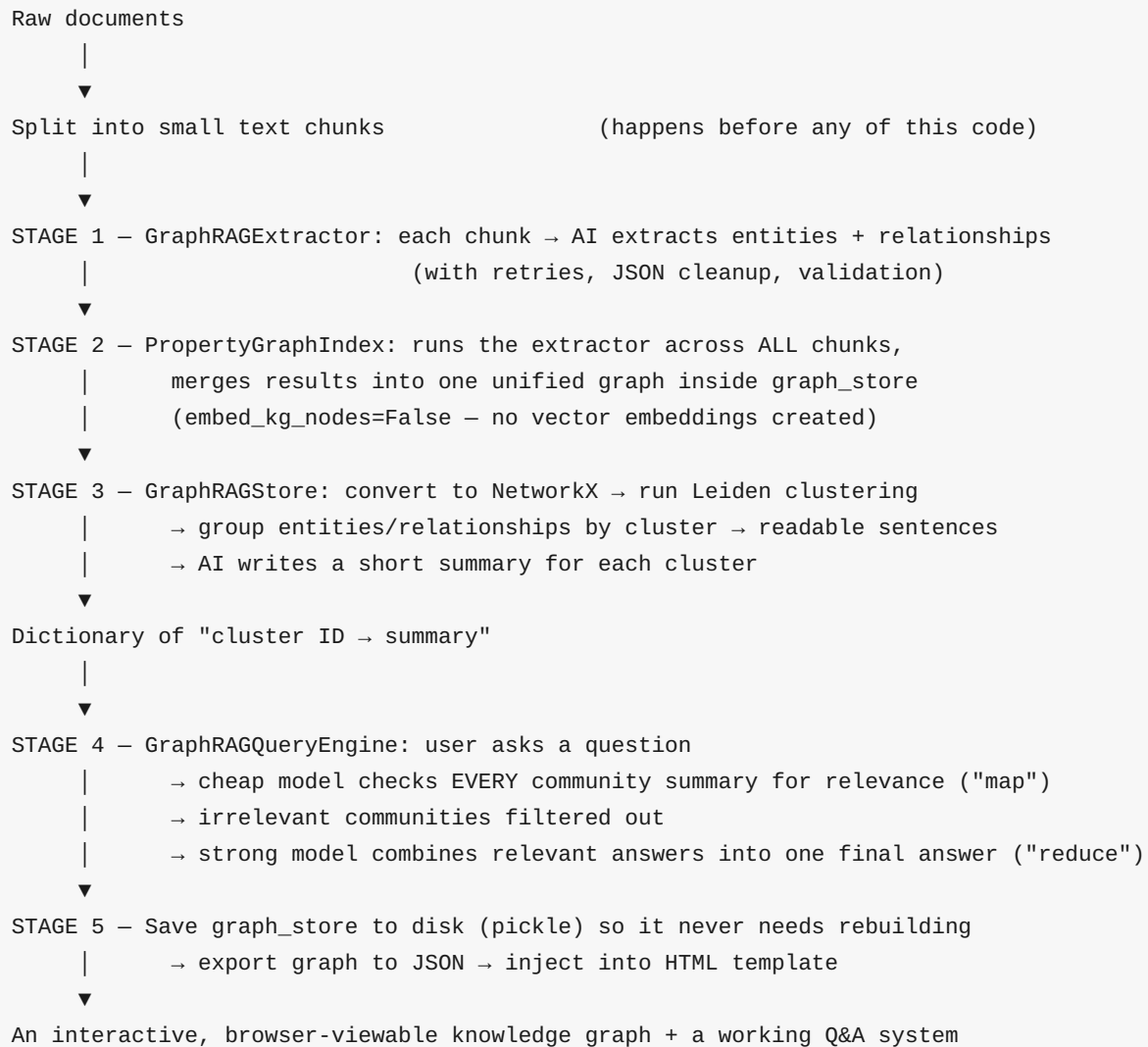
    html = pathlib.Path(template_file).read_text()
    html = html.replace("GRAPH_DATA_PLACEHOLDER", json.dumps(graph_data))
    pathlib.Path(output_file).write_text(html)
```

Two ways to call it: pass a live `graph_store` (exports fresh JSON first), or leave it out to just reuse a previously saved JSON file — same "don't redo expensive work" philosophy as the pickle step.

The actual trick is simple **string substitution**: an existing HTML template file contains a literal placeholder (`GRAPH_DATA_PLACEHOLDER`) somewhere inside a `<script>` tag. This code finds that placeholder text and replaces it with the real graph data (as a JSON string), then saves the result as one self-contained `.html` file. Since D3.js visualizations run in the browser using JavaScript (not Python), this is how Python hands off the finished data to something that can actually draw and animate it.

Opening the resulting file gives an interactive graph: color-coded by entity type, clickable to highlight neighborhoods, searchable, with sliders and tooltips — all powered by the JSON data now baked into the page.

The Whole Journey, Start to Finish



Why This Design Matters — Summary Table

Problem	How it's solved
AI doesn't always follow format instructions	Retry loop + markdown-fence cleanup + double validation
API rate limits	Sleep between calls + rotating API keys
One bad chunk/community shouldn't break everything	Failures are logged and skipped, not fatal
Need to trace facts back to their source	Source metadata stamped on every entity/relationship
Hundreds of disconnected facts are hard to use	Clustering + summarization turns them into readable topics
Too many summaries to fit in one prompt	Map-reduce: check each community individually, then combine only relevant ones
Answering questions shouldn't be expensive	Cheap model for repeated per-community checks, strong model only for final synthesis
Rebuilding the graph is slow and costly	Pickle the finished graph_store to disk; reload instantly next time
Need a way to actually <i>see</i> the graph	Export to JSON → inject into a D3.js HTML template for an interactive view